Problem Set 2

1) a) We have n/k sublists each of length k. In worst case, each of these sublists are reverse sorted, and insertion sort on each ~~ta~~ hence takes time $\Theta(k^2)$. We have to do this operation on n/k sublists, thus getting $n/k \times k^2 = nk$ and hence total time $\Theta(nk)$

b) At each level of the recursion tree, the merge function merges each pair of sublists in total ~~of~~ $\Theta(n)$ time. This process happens at each level, and there are a total of ~~n/k~~ $\log(n/k)$ levels, thus getting a runtime of $\Theta(n) \times \Theta(\log(n/k))$
$= \Theta(n \log(n/k))$

c) We need $nk + n\log(n/k) = \Theta(n\log n)$
$k + \log(n/k) = \Theta(\log n)$

Since $\log(n/k) \leq \log(n)$, the LHS is at least k* and at most $k + \log(n)$. So ~~for it to~~ k must be $\Theta(\log n)$, because otherwise ~~then running time~~ k will increase at a faster rate than $\log(n)$ and thus ~~have~~ the running time will be slower than standard merge sort in terms of Θ-notation.

Minor note:- Substituting into $\log(n/k) \Rightarrow \log(n) - \log(c \log n)$
$= \log n - \log(c) - \log(\log(n))$
$= \Theta(\log n)$

d) In practice k should be the largest possible length ~~for~~ of an array ~~/list etc~~ for which insertion sort is faster than merge sort

2) a) A' contains the exact same elements as A, ~~no more and no less~~ without any extras or any missing.

b) Loop invariant:- At the start of each iteration, A[j] is the smallest element in the subarray A[j:n]

Initialization:- At the beginning of the ~~for~~ first ~~first~~ iteration ~~loop~~, j==n and so the subarray A[j:n] = A[n:n] which has only one element hence the invariant is true.

Maintenance:- During any iteration of the for loop, we check if the element ~~to the left of~~ at A[j-1] is less than A[j]. If it is, then their positions in the array are replaced and so A[j-1] now holds the smallest element ~~If not, then A[j-1] is smaller than A[j]~~ in A[j-1:n]. If not then A[j-1] still holds the smallest element in A[j-1:n]. Before the next iteration, j is decremented by 1 (so j=j-1) and so the loop invariant holds true.

~~(1)~~ ~~H.3)~~

Termination :- The loop terminates when $j = i$ and so $A[i]$ ~~be~~ is hence the smallest element in $A[i : \cancel{1} n]$ (by the invariant).

c) Loop invariant :- At the start of each iteration of the first loop, the subarray $A[\cancel{1} 1 : i-1]$ contains the smallest $i-1$ elements of A in sorted order.

Initialization :- Before the first iteration, $i = 1$ and so $\cancel{\#} A[1 : i-1] = A[1 : 0]$ which is empty and so the invariant is vacuously true.

Maintenance :- Assume the invariant is true before an iteration. Then, ~~of~~ by part (b), $A[i]$ is the smallest element in $A[i \cancel{:} : n]$ after an iteration, ~~and before the next iteration, i is incremented by 1~~ (thus $A[1 : i]$ contains the smallest i elements of A in sorted order. Before the next iteration, i is incremented by 1 and thus the invariant holds.

Termination :- The loop terminates when $i = n$ and so $A[1 : n-1]$ has the smallest $n-1$ elements of A in sorted order, ~~Thus~~ ~~Si~~ hence $A[n]$ being the last element is the greatest and thus already in the correct position, so the array A is sorted.

d) For every iteration of the outer for loop, the inner loop runs ~~E~~ $n-i$ times where each iterations takes constant time. So the total iterations of the inner loop is:-

$$(n-1)+(n-2)+(n-3)+\cdots+(n-(n-1))$$

$$= \sum_{i=1}^{n-1}(n-i) = \frac{n(n+1)}{2}$$

$$= \frac{n^2+n}{2}$$

$$= \Theta(n^2)$$

The worst and best case running time of bubble sort is $\Theta(n^2)$ wheres for insertion sort, ~~it is~~ the ~~bes~~ worst case running time is $\Theta(n^2)$ but the best case is $\Theta(n)$.

3) a) The ~~in~~ for loop runs ~~n~~ $n+1$ times, with each iteration ~~cos~~ taking constant time and all the other code taking constant time. Hence the runnig time is $\Theta(n)$.

b)

```
POLYNOMIAL (A, n, x)
p = 0
for i = 1 to n
    z = 1
    for j = 1 to i
        z = z x x
    p = ~~(A[i] x z)~~ p + (A[i] x z)
return p
```

The inner loop runs $\sum_{i=0}^{n} i$ ~~[illegible]~~ times, which evaluates to $\frac{n(n+1)}{2}$

$$= \frac{n^2+n}{2}$$

$$= \Theta(n^2)$$

with each iteration taking constant time

c) Initialization:- Before the first iteration, the value of i is n, so the invariant is $p = \sum_{k=0}^{n-(i+1)} A[k+i+1] \cdot x^k$. Since $n-(i+1) = n-i-1 = n-n-1 = -1$, the summation has no terms and so $p = 0$, which is true.

Maintenance:- Suppose the invariant is true before an ~~certain~~ iteration with index i. During the iteration:-

$$p = A[i] + x \times \sum_{k=0}^{n-(i+1)} A[k+i+1] \times x^k$$

$$p = A[i] + \sum_{k=0}^{n-(i+1)} A[k+i+1] \times x^{k+1}$$

Let $j = k+1$. Then:-

$$p = A[i] + \sum_{j=1}^{n-i} A[j+i] \times x^{j}$$

$$p = \sum_{j=0}^{n-i} A[j+i] \times x^{j}$$

$$p = \sum_{k=0}^{n-i} A[k+i] \times x^{k}$$ (Since j is just a placeholder)

At the start of the next iteration, i is decremented by 1, so $i_{new} = i_{old} - 1 \Rightarrow i_{old} = i_{new} + 1$.
Substituting we get the loop invariant:-

$$p = \sum_{k=0}^{n-(i+1)} A[k+i+1] \times x^{k}$$ hence invariant maintained.

Termination:- The loop terminates when $i = -1$, plugging into the invariant we get:-

$$p = \sum_{k=0}^{n} A[k] \times x^{k}$$

4) a) $(1,5)$, $(2,5)$, $(3,5)$
$(4,5)$, $(3,4)$

b) An array sorted in descending order. It has $\frac{n(n-1)}{2}$ inversions

c) Let $A[1 \dots N]$ be any array and let $inv(i)$ denote the number of inversions (j, i) in A.

The while loop in insertion sorts moves the element at index i $inv(i)$ positions to the left and gets rid of $inv(i)$ inversions in the process. This while loop runs for each value of i from 2 to N ~~so~~ a total of:

$$I = \sum_{i=2}^{n} inv(i)$$

times, which is the total number of inversions and so the running time is $\Theta(n + I)$

~~d) The merge procedure provided is kept as is. However for the first 11 lines. The f. and the lines~~

d)

```
MERGE (A,p,q,r)
  *Lines 1 to 11 kept as provided*
  inv = 0
  while i < nL and j < nR
        if L[i] ≤ R[j]
            A[k] = L[i]
            i = i+1
        else
            A[k] = R[j]
            j = j+1
            inv = inv + (nL - i)
        k = k+1
  *Lines 19 to 27 kept as provided*
  return inv

MODIFIED-MERGE-SORT (A,p,r)
  if p ≥ r
     return 0
  inv = 0
  q = ⌊(p+r)/2⌋
  inv = inv + MODIFIED-MERGE-SORT (A,p,q)
  inv = inv + MODIFIED-MERGE-SORT(A,q+1,r)
  inv = ~~MER~~inv + MERGE (A,p,q,r)
  return inv
```